

1 Web 的基本结构与工作过程

1.1 Web 服务器和浏览器

Web 服务器可以分布在 Internet 的任意位置，每个 Web 服务器都保存着可以被 Web 浏览器共享的信息。这些信息通常以页面（Web 页面）的方式进行组织，页面一般是超文本/超媒体文档，页面间通过**超链接**建立连接。

万维网是分布式超媒体（hypermedia）系统，它是超文本（hypertext）系统的扩充。超媒体与超文本的区别是文档内容不同。超文本文档仅包含文本信息，而超媒体文档还包含其他表示方式的信息。

1.2 IP 和域名

DNS-over-TLS（DoT）类似于 HTTP-over-TLS（HTTPS），是一种基于 TLS 来进行报文加密的 DNS 请求交互协议。

DNS over HTTPS（缩写：DoH）是一个进行安全化的域名解析的方案。其意义在于以加密的 HTTPS 协议进行 DNS 解析请求，避免原始 DNS 协议中用户的 DNS 解析请求被窃听或者修改的问题。

和 DoT 试用 853 特定端口不同，DoH 使用 HTTPS 通用端口 443，更难以检测。

1.3 Web 标准

Web 标准并不是某一个标准，而是一系列标准的集合，主要包括：

结构（Structure）标准：HTML、XML、XHTML

呈现（Presentation）标准：CSS

行为（Behavior）标准：DOM、ECMAScript

1.4 小结

万维网是互联网的一部分，是由英国科学家蒂姆·伯纳斯李(Tim Berners-Lee)于 1989 年在欧洲核子研究组织(CERN)发明的。

万维网的基本结构是采用开放式的客户/服务器结构（Client/Server），分成 Web 服务器（服务器端）、Web 浏览器（客户端）以及传输协议（HTTP 协议）三个部分。

Web 是超文本和互联网两大技术融合的结晶。Web 技术的核心是三个协议：

- 统一资源识别器(Uniform Resource Locator,URL)，解决文档（这个概念被扩展为资源）的命名和寻址

- 超文本传输协议(Hypertext Transfer Protocol, HTTP)，解决文档的快速传输

- 超文本标记语言 HTML(Hypertext Markup Language, HTML)，解决超文本文档的表示。

2 超文本标记语言 HTML

Web 标准并不是某一个标准，而是一系列标准的集合，主要包括结构（Structure）、呈现（Presentation）和行为（Behavior）三个方面。

2.1 HTML 标签/HTML 元素

2.1.1 定义

一个完整的元素（Element）= 开始标签 + 闭合标签 + 内容。

标签大小写不敏感，推荐使用小写。

存在自闭合元素（如<meta/>），没有结束符号，没办法在内部插入其他标签或文字，只能定义属性。元素可以有属性（Attribute），一个属性应该包含：

- 在属性与元素名称（或上一个属性，如果有超过一个属性的话）之间的空格符；

- 属性的名称，并接上一个等号；

- 由引号所包围的属性值，单双引号均可。

注意，存在没有写属性值的属性，称为布尔属性。布尔属性合法的属性值只有空字符串（等价于不写）和与属性名同名的字符串两种，其它都是非法的。

2.1.2 块级元素 vs 内联元素

块级元素（block level element）在页面中以块的形式展现——相对与其前面的内容它会出现在新的一行，其后的内容也会被挤到下一行呈现。（按行为列，一块一块依次往下排列）

块级元素通常用于展示页面上结构化的内容，例如段落、列表、导航菜单、页脚等等。块级元素不会被嵌套进内联元素中，但可以嵌套在其它块级元素中。常见的块级元素有：

- 段落 Paragraphs (<p>)
- 标题 Headers (<h1> -<h6>)
- 分区 Divisions (<div>)
- 列表及列表项 Lists and list items(, ,)
- 表单 Forms (<form>)

内联元素（inline element）通常出现在块级元素中并包裹文档内容的一小部分，而不是一整个段落或者一组内容。内联元素又称为行内元素。

内联元素不会导致文本换行：它通常出现在一堆文字之间。例如超链接元素<a>或者强调元素和。常见的内联元素有：

- Spans ()
- Images ()
- Anchors (<a>)

2.1.3 HTML 文档结构

一般的结构为：

1. 文档声明。声明不是 HTML 标签，它是指示 web 浏览器关于页面使用哪个 HTML 版本进行编写的指令。`<!DOCTYPE html>`是**最短的有效**的文档声明。

2. `<html>`元素，包裹了整个完整的页面，是一个根元素，所有其他元素必须是此元素的后代。`<html>`与`</html>`标签限定了 HTML 文档的开始点和结束点，在它们之间是文档的头部和主体，文档的头部由`<head>`标签定义，而主体由`<body>`标签定义。

`<head>` 元素用于定义文档的头部，它是所有头部元素的容器。`<title>`元素定义文档的标题，它是`<head>`元素中唯一必需的元素。

`body` 元素定义文档的主体。`body` 元素包含文档的所有内容。

2.1.4 HTML 文档头部

(1) 添加网页标题：`<title>`元素。`<title>北京邮电大学</title>`

(2) 添加网页元信息：`<meta>`元素。通过 `name` 属性定义作者(author)、关键词(keywords)、描述(description)、版权(copyright)等。如：`<meta name="description" content="北京邮电大学是一所...">`。

同时，也可以用 `name` 属性值控制爬虫策略。如果需要设置全站爬虫交互策略，使用 `robots.txt` 文件。

还可以通过 `http-equiv` 属性回应给浏览器一些有用信息，帮助正确和精确地显示网页内容。例如：

`<meta http-equiv="Refresh" content="5;url=http://www.example.com" />`，5 秒钟后将被重定向到 `www.example.com`，用于网站搬家。

(3) 使用`<link>`元素为站点添加自定义图标，图标出现在浏览器标签页以及书签中。例如，`<link rel="shortcut icon" href="favicon.ico" type="image/x-icon">`

(4) 应用 CSS 和 JavaScript。

方式 1，利用元素引用外部 CSS 样式文件：

```
<link rel="stylesheet" href="my-css-file.css">
```

方式 2，利用`<style>`元素定义 CSS 样式：

```
<style type="text/css">
```

```
...// 在这写 css 定义
```

```
</style>
```

对于 javascript:

方式 1，利用`<script>`元素引用外部 js 脚本文件：

```
<script src="my-js-file.js"></script>
```

方式 2，直接将脚本放入< script>元素中，而不是指向外部脚本文件：

```
<script type="text/javascript">
    ...// 在这写 Javascript 代码
</script>
```

2.1.5 HTML 文档主体

(1) 段落与文字

通过标题元素<h1>~<h6>定义多级标题。<h1>-<h6>是有序的，一般<h1>只有一个。

利用<p>元素定义段落，利用
元素在不产生新段落的情况下进行换行。注意，段落标签会自动换行，
标签是给文字换行，**不会带来空隙**；<p>标签用来给文字分段，**段落间有空隙**。

 (emphasis) 元素用于标记需要用户着重阅读的内容；

 (strong importance) 元素用于标记非常重要的内容；

<sup>和<sub>元素用于标记上标和下标；

<time>元素将时间和日期标记为可供机器识别的格式；

<abbr>元素用于标记缩略语并在光标移动到上面时提供解释；

<address>元素用于标记联系方式；

<hr/>(horizon)元素用于标记水平线；

<code>用于标记计算机通用代码；

<pre>用于保留空格（通常是代码块）。如果不使用，最多渲染出一个空格；

<var>用于标记具体变量名；

还有块级引用：如果一个块级内容（一个段落、多个段落、一个列表等）在其他地方被引用，应把它用<blockquote>元素包裹起来，在其 cite 属性中用 URL 指向引用资源的位置，可以为<blockquote>施加额外的样式。

对应的，利用<q>元素插入不需要分段的短引用，同样修改其 cite 属性。

<div> 元素是块级元素。没有特定含义，提供组合其他 HTML 元素的**容器**，包裹的元素为可以放到 body 元素内的任何元素。<div>常见用途有两种：一是与 CSS 一同使用，为大块内容设置样式属性，二是用于文档布局。

对应的，** 元素是内联元素。**同样没有特定含义，可以使用 来组合内联元素，以便通过样式来格式化它们。可以为 span 设置 id 或 class 属性，这样既可以增加适当的语义，又便于对 span 应用样式。

利用<!--和-->把需要注释的内容包起来。

特殊字符，如<、>、”，等等，有特别的等价字符引用方式。

(2) 列表

一共有三种：

- 有序列表(ordered list,)
- 无序列表(unordered list,)
- 描述列表(description list, <dl>)

无序列表和有序列表中的列表项是(list item)。而描述列表中的列表项是<dt> (description term)，描述内容是<dd> (description description)。

举个例子吧：

咖啡

牛奶

茶

这是一个有序列表：

咖啡

牛奶

茶

这是一个描述列表：

<dl>

<dt>咖啡</dt><dd>我不爱喝咖啡...</dd>

<dt>牛奶</dt><dd>我爱喝牛奶...</dd>

<dt>茶</dt><dd>我爱喝红茶...</dd>

</dl>

此外，可以设置有序列表的编号。修改的属性 start，允许从 1 以外的数字开始计数；reversed 属性将启动列表倒计数。如<ol start=4 reversed>。

注意，元素是块级元素，意味着其中可以包含任意元素。

(3) 表格

表格由<table>元素定义。

表格的行由<tr>元素(table row)定义。

每行被分割为若干单元格，单元格由<td>元素定义(table data)。有几个 td 元素就有几列。

单元格可以包含文本、图片、列表、段落、表单、水平线、表格等等。

<caption>元素用来定义表格标题，默认在表格里的第一行，且唯一。

<th>(table header)元素用来定义列和行的表头单元格，浏览器默认会以粗体和居中的样式显示元素中的内容。

<th>和<td>有 colspan 和 rowspan 属性，允许单元格跨越多行和多列（合并单元格）。

还可以添加<thead>，<tfoot>，<tbody>元素，给表格分成不同结构。

（4）多媒体与嵌入

利用元素嵌入图片。属性 src 为图像地址；属性 alt 为图像无法显示时的提示文字；属性 title 为鼠标移到图片上的提示文字。src 和 alt 是必须的，title 往往和 alt 相同。搜索引擎主要依赖 alt 属性判断图片含义。

利用<figure>和<figcaption>元素，为图片提供语义容器：

<figure>

<figcaption>北京邮电大学 logo</figcaption>

</figure>

<figure> 里面里不一定要是一张图片，可以是几张图片、一段代码、音视频、方程、表格或别的。

注，网页图片格式分为 2 种：位图，矢量图。矢量图与分辨率无关，色彩不丰富；位图是由一个一个像素点产生，色彩丰富，但当放大图像时，像素点也放大了，从而影响清晰度。网页中常见的位图有 jpg（文件较大）、png（无损压缩）、gif。

利用<video></video>元素嵌入视频。还有别的属性，控制循环播放（loop）、显示控件（controls）、马上播放（autoplay）等。可以插入多个<source>标签解决多格式支持问题，浏览器将会检查<source> 标签，并且播放第一个与其自身解码程序相匹配的媒体。

<audio>除 weight/height/poster 等视觉相关属性外，支持所有 video 拥有的属性，用来插入音频。

利用<embed>和<object>元素实现通用嵌入，如 embed 嵌入 flash，object 嵌入 pdf 等。

利用<iframe>元素将另一个 HTML 页面嵌入到当前页面中。Src 属性是 iframe 的必需属性。

（5）超链接

利用<a>元素插入超链接。基本语法：

可以使用 target 属性控制目标窗口打开方式，如打开新窗口等。

超链接可以链接到 html 文档的指定位置（被称为文档片段），点击链接可以跳转到指定位置。比如：

```
<h2 id="h2addr"></h2>
```

```
<a href="index.html#h2addr"></a>
```

在 HTML5 中，可以通过<a>元素包括 div、h 元素、段落标记 p 等块级元素，创建**块级链接**。不能在一个 a 标签内包含另一个 a 标签，随浏览器不同而产生未知效果。

注：src 和 href 的区别：

href（常用有 link、a）用于在当前文档和引用资源之间建立联系；

src（常用有 img、script、iframe）用于将资源嵌入到当前文档元素位置。当浏览器解析到该元素时，在浏览器下载、编译、执行这个文件之前，页面的加载和处理会被暂停。

（6）表单 form

```
<form>
```

- 利用 input 元素插入**文本框**；
- 利用 select、option、optgroup 元素插入下拉列表；
- 利用 textarea 元素插入多行输入字段；
- 利用 button 元素插入点击按钮；
- 等等。以上元素可以被 div 包裹。

```
</form>
```

action 规定当提交表单时处理表单发送数据的程序的 URL。

name 规定表单名称。

method 规定提交表单时的 HTTP 方法，GET（默认）或 POST。

在所有表单元素中都有一组通用属性，可以对小部件进行控制。通用属性包括 disabled（是否能与用户交互）、value（元素初始值）等。

下面先着重描述 input。<input>元素很特别，几乎可以是任何东西，通过简单设置 type 属性（<input type=" " >），它可以彻底变成大多数类型的表单小部件，包括单行文本字段、没有文本输入的控件、时间和日期控件和按钮等。

所有输入框都有一些通用规范：

可以被标记为 readonly（不能改）或者 disabled（改了也发不出去）；placeholder 属性，输入框中出现的文本；被限制在 size 和 maxlength 之中。

输入框可以是文本框 text：

```
<input type="text" value="默认文字" size="123" maxlength="123"/>
```

也可以是密码文本框 password：

```
<input type="password" value="默认文字" size="123" maxlength="123"/>
```

区别在于是否明文显示。

input 中类型有单选按钮 radio。单选按钮 radio 只能从选项列表中选一项。

```
<input type="radio" name="abc" value="单选按钮的取值"/>
```

name 和 value 是 radio 的必要属性。对于同一个问题的不同选项要设置一个相同的 name，才能将选项归到一个问题上。

复选框 checkbox。

```
<input type="checkbox" name="abc" value="复选框取值" checked="checked"/>
```

checked 属性表示该选项在默认情况下已经被选中。name 和 value 是 checkbox 的必要属性。HTML 中的复选框是没有文本的，需要加入 label 标签，并且用 label 的 for 标签指向复选框的 id。如：

```
<input id="cb1" type="checkbox" checked="checked"/>
```

```
<label for="cb1">苹果</label>
```

还可以表示按钮。一共有三种：普通按钮 button、提交按钮 submit、重置按钮 reset。对于 button 而言：

```
<input type="button" value="按钮上的文字" onclick="JS 脚本程序"/>
```

其他两种按钮类似。注意，submit 和 reset 只针对当前 form 标签内的表单内容有效。

Input 元素还可以有图片 image，只是没啥别的特殊动作。

```
<input type="image" src="。。。"/>
```

终于说完了上面的 input 元素。下面讲一讲别的。

利用<textarea>元素插入多行文本输入控件。

```
<textarea rows="行数" cols="列数">多行文本框</textarea>
```

利用<select>和<option>元素插入下拉选择框。下拉列表需要由这两个标签配合使用。利用<optgroup>设置选项分组：

```
<select id="groups" name="abc">
```

```
  <optgroup label="fruits">
```

```
    <option>Banana</option>
```

```
    <option selected>Cherry</option>
```

```
  <optgroup label="vegetables">
```

```
    <option>Carrot</option>
```

```
</select>
```

还可以利用<datalist>元素插入自动补全输入框。自动补全输入框提供一个下拉框向用户展示的，在输入框中输入可能匹配的内容。option 负责指定值，list 属

性将数据列表绑定到一个文本域（通常是一个 input 元素）。

```
<input type="text" name="abc" id="12" list="mySuggestion">
<datalist id="mySuggestion">
  <option>Apple</option>
  <option>Banana</option>
</datalist>
```

可以使用<fieldset>和<legend>元素创建控件组。Fieldset 进行分组，一个表单可以有多个控件组；legend 用于对每组内容进行描述。相当于是把他们框起来了，外圈有一线框。

```
<form>
  <fieldset>
    <legend>Something to say.</legend>
    <input type="radio" id="r1"><label for="r1">HTML</label></br>
    <input type="radio" id="r2"><label for="r2">CSS</label></br>
    <input type="radio" id="r3"><label for="r3">Javascript</label></br>
  </fieldset>
</form>
```

在需要的时候，使用 label 元素为 input 元素（如 radio）设置标签。

2.1.6 文档与网站机构

网页的基本组成部分：

页眉<header>、导航栏<nav>、主内容<main>、侧边栏<aside>、页脚<footer>。

在主内容 main 中，可以包括各种内容，如 article、section、div 等。

在需要时，分内联和块级元素，使用 span 和 div 标签。

2.1.7 HTML 小结

```
<!DOCTYPE html>
<html>
<head>
  添加网页标题：title 元素
  添加网页元信息：meta 元素
  应用 css 和 Javascript：link 和 script 元素
</head>
<body>
  段落和文字：p、h1、div、em、strong
  列表：<ul><ol><dl>，<li><di><dd>
```

表格: <table><tr><td>, <th><caption>, <thead><tfoot><tbody>

多媒体嵌入: <video><audio>, <embed><object>, iframe

超链接<a>

表单<form>, <input><textarea><select><datalist><fieldset><legend><label>
</body>
</html>

学习 HTML 的重点不在于掌握了多少标签,而在于掌握标签的语义,知道如何编写一个语义结构良好的页面。

3 层叠样式表 CSS

Css 三种引用方式:

(1) 外部样式表 (link 引入): <link rel="stylesheet" href="style.css">

(2) 内部样式表 (在 head 中的 style 标签内定义):

<style> p { color: blue; } </style>

(3) 内联样式 (在元素的 style 属性上定义): <p style="color:red;"></p>

CSS 规则 = 选择器 + 声明块。

CSS 对大小写不敏感,但是选择其中的 class 和 id 名称对大小写敏感,推荐全小写。

3.1 选择器

可以划分为:

(1) 简单选择器

1. 元素选择器。选中所有元素名匹配的元素。不区分大小写。比如 p、div。
2. 标识(id)选择器: “#” 开头。
3. 类选择器: “.” 开头。

(2) 属性选择器

根据元素的属性和属性值来匹配元素。通用语法如下:

a[href]、a[href="https://..."]、*[lang="en"]

匹配方式一共有两种:

1. 根据属性的存在或属性值进行精确匹配。比如:

[attr]: 选择带有 attr 命名的属性的元素

[attr=value]: 选择 attr 值为 value 的元素

[attr~value]: 选择带有 attr 的属性,且该属性是一个以空格分隔的值列表,至少有一个值为 value。

2. 根据属性值的子串进行类正则表达式的非精确匹配。比如:

[attr=val]: 选择 attr 属性的值为 val, 或以 “val-” 开头 (这是短划线);

[attr^=val]: 选择 attr 属性的值以 val 开头 (包括 val);

[attr\$=val]: 选择 attr 属性的值以 val 结尾 (包括 val);

[attr*=val]: 选择 attr 属性的值包含子字符串 val。

(3) 伪类 (处于确定状态的元素, 如鼠标指针悬停的元素)

以冒号 (:) 为分割, 添加到选择器末尾, 选择**特定状态下的元素**。

1. 状态伪类:

:link 伪类表示链接的正常状态, 选择那些尚未被点过的链接;

:visited 伪类选择点过的链接;

:focus 用于选择已经通过指针设备、触摸或键盘获得焦点的元素, 在表单里使用得非常多;

:hover 伪类在用户指针悬停时生效, 不只适用于链接;

:active 伪类选择被鼠标指针或触摸操作“激活的”元素, 只发生在鼠标被按下到被释放的这段时间里。

2. 结构伪类

:first-child 伪类选择是父元素第一个子元素的指定类型元素;

:last-child 伪类选择是父元素最后一个子元素的指定类型元素;

:nth-last-child(n) 选择是父元素倒数第 n 个子元素的指定类型元素。

子、父元素可以理解为在一个标签/容器内的关系。

其他不一一列举。

3. 验证伪类

:in-range 伪类选择有范围且值在指定范围内的元素, 反面:out-of-range

:disabled 伪类选择禁用状态的表单元素, 反面:enabled

4. 语言伪类

:lang() 基于元素语言来匹配页面元素。如 li:lang(“en”)

(4) 伪元素 (处于确定位置的元素, 如每个段落第一个字)

以双冒号 (: :) 为分割, 添加到选择器末尾, 选择**元素的某个特定部分**。

1. ::before 与 ::after

为其他 HTML 元素添加内容 (文本或图形), 添加的内容不实际存在于 DOM 中, 但可以像存在一样操作它们, 需要在 CSS 中声明 content 属性。

2. ::first-letter 伪元素选择一行文本第一个字符, 并且文字所处的行之前没有其他内容 (如图片和内联的表格), 只适用于块级元素, 内联元素不适用。

3. ::first-line 选择元素的第一行, 只适用于块级元素。

4. ::selection 选择文档中被高亮选中的部分。

(5) 组合选择器（组合多个选择器，实现特定选择）

1. 群组选择器（A, B）。匹配满足 A 和 B 的任意元素；
2. 后代选择器（A B）。匹配任意满足：B 是 A 的 DOM 树上的后代节点；
3. 子元素选择器（A>B）。匹配任意满足：B 是 A 的直接子节点；
4. 相邻兄弟选择器（A + B）。匹配任意满足：B 是 A 的下一个兄弟结点；
5. 后继兄弟选择器（A~B）。匹配任意满足：B 是 A 的兄弟结点的任意一个。

注意：星号（*）是通配选择器，匹配任意的 HTML 元素。

(6) 多重选择器（多个选择器设置相同规则）

小结：解读 CSS 选择器——从右往左的识别。

3.2 样式

3.2.1 文本和段落样式

一般有两类：文本的展示效果，设置字号、字体、阴影等；文本的布局风格，设置文本的间距、其他功能等。

(1)font-family 属性，可指定多个字体。

font-family: 字体 1,字体 2,字体 3;

(2)font-size 属性，字体大小。设置绝对大小或相对大小（相对父元素），或使用像素 px 或 em 数字值(倍数)、百分比等。

font-size: 关键字/值;

(3)font-weight, 字体粗细。设置关键字 normal、bold 或直接给数值 100、200。

(4)font-style, 字体样式。属性有 normal、italic（斜体文字）、oblique（让没有斜体属性的文字倾斜）。

(5)color, 颜色。有以下几种表示形式：

- 十六进制颜色。格式为#RRGGBB，如，background-color: #0000ff;
- RGB 颜色。指定 RGB(红，绿，蓝)，值在 0-255 或 0-100%之间；
- RGBA 颜色。指定 RGBA(红，绿，蓝，alpha)，即透明度，介于 0-1；
- HSL 颜色。指定 HSL(色相 0-360，饱和度 0-100%，明度 0-100%);
- HSLA 颜色。HSLA(色调，饱和度，亮度，alpha)。同上，增加透明度；
- 预定义/跨浏览器的颜色名称。

(6)text-decoration, 文本装饰。包含 none、underline（下划线）、line-through（删除线）、overline（顶划线）。可以同时有多个属性，空格分隔。

(7)text-transform, 文本转换。包含 none, uppercase（全部大写），lowercase（全部小写），capitalize（首字母大写）。

(8)text-shadow, 文本阴影。语法为：text-shadow: 水平偏移 垂直偏移 模糊

半径 颜色；比如，text-shadow: -4px 4px 5px red; 可通过包含以逗号分隔的多个阴影值，将多个阴影应用于同一文本。

(9)text-indent, 文本缩进。定义段落的首行缩进，text-indent: 属性值；缩进值可以是负数。

(10)text-align, 文本对齐。只针对文本文字和 img 标签有效。包含 left、right、center、justify (使文本展开，改变单词之间的差距，使所有文本行的宽度相同)；

(11)line-height, 行高。Line-height: 属性值；

(12)letter-spacing, 字母间距。word-spacing, 词间距。二者的属性可以包含：normal、length、inherit；

3.2.2 列表样式

在无序列表或有序列表上可以通过以下属性改变列表样式：

- list-style-type, 设置列表项前符号/数字
- list-style-position, 设置列表项符号位置
- list-style-image, 设置列表项符号为图片

三个值可以缩写成一个属性：list-style。属性值可以任意顺序排列。

3.2.3 链接样式

链接有几种状态：Link 未访问、Visited 已访问、Hover 悬停、Focus 被选中、Active 被激活如点击。

使用伪类进行设置。这些状态的定义/设置需要按照以上的顺序进行：lvhfa。

鼠标样式设置：cursor 属性

为链接添加图标：background: url("abc.png");

利用列表和链接做出导航菜单：将元素设置成 display: inline;

3.2.4 图片样式

width 和 height 定义图片宽度、高度；

border 定义边框，包含 border-width、border-style、border-color 三个属性，可以简写成 border: 1px solid gray;

text-align 在图片的父元素中，定义水平对齐方式；

vertical-align 定义垂直对齐方式，只对内联元素、表格单元格元素生效；

float 定义文字环绕效果，能应用于所有的元素。包含 left 向左浮动、right、none、inherit。

3.2.5 背景样式

Background-color, 背景颜色；Background-image, 定义背景图像的路径；

Background-repeat, 定义背景图像显示方式，如纵向平铺，横向平铺，默认在水

平和垂直方向同时平铺；Background-position，背景图像在元素的起始位置，给两个参数表示坐标；Background-attachment，定义背景图像是否随内容而滚动。比如滚动条滚动，该图像是否一起动，默认一起动。

3.2.6 表格样式

Border-collapse，表格边框合并。合并则一条边框，否则两条。默认不合并；

Border-spacing，表格与边框间距。只有一个属性值，会同时作用于横向和纵向间距；有两个属性值，则第 1 个用于横向，第 2 个用于纵向；

3.3 层叠和继承

3.3.1 层叠

CSS 将根据选择器的权重，选择使用权重高的选择器，低的则被覆盖。

层叠优先级算法：

1. 针对某一元素的某一属性，列出所有给该属性指定值的 CSS 规则；
2. 根据声明的重要性和来源进行优先级排序。如重要声明!important 与一般声明，css 规则来源排序如下（从低到高）：

浏览器默认样式 -> 用户样式表的普通声明 -> 作者样式表的普通声明 -> 作者样式表的重要声明 -> 用户样式表的重要声明

3. 根据选择器的特殊性排序。使用四种不同的值来度量，可被认为是千位、百位、十位、个位。

千位：如果声明是在标签的 style 属性中，该列加 1 分，否则为 0；

百位：在选择器中每包含一个 ID 选择器就在该列中加 1 分；

十位：在选择器中每包含一个类、属性、伪类选择器就在该列中加 1 分；

个位：在选择器中每包含一个元素选择或伪元素选择器就在该列中加 1 分；

注意，通用选择器*、组合选择器(+,>,<,~,)、否定伪类(:not)不影响特殊性。

4. 根据 CSS 规则在样式源文档中出现的位置进行优先级排序。

以上四条规则在冲突时才会往下应用。如果有多条设定都应用，不冲突属性值直接应用，冲突属性值通过优先级算法解决。

3.3.2 继承

子元素可以继承父元素的属性。以避免同样的内容重复声明。可以通过属性 inherited 控制是否继承。通常文本相关属性会默认继承，布局相关属性不默认继承。如果继承属性有冲突，则 DOM 树中离子孙最近的那个祖先的属性值会被继承下来。

all 属性：用于重置或继承元素的所有属性，包含 inherit、initial、unset；

- **inherit**: 所有样式属性将从父元素继承。如果父元素没有指定这些属性，浏览器将使用默认样式；

- **initial**: 所有样式属性将重置为其初始值，即浏览器的默认样式；

- **unset**: 此值将属性设置为 **inherit** 或 **initial**，取决于属性是否继承。非继承属性（如 **background-color**）将重置为初始值，继承属性（如 **color**）则会继承自父元素；

3.3.3 小结

样式覆盖的 5 种方式：

（1）引用方式冲突。利用特殊性进行比较；

（2）继承方式冲突。DOM 树上最近的祖先获胜；

（3）指定样式冲突。根据特殊性比较；

（4）继承和指定样式冲突。指定样式获胜；

（5）**!important**。重要声明会覆盖其他一般声明。如果多个重要声明冲突，仍看特殊性来比较。

3.4 盒模型

盒模型包括：内容 content、内边距 padding、边框 border、外边距 margin。

（四个主要属性，由内依次向外）

Content 是必须的，其他三部分是可选的。有三个属性，width、height、overflow。Overflow 可以用来指定溢出的处理方法。注意，width 和 height 只针对内容区，且只有块级元素可以设置 width 和 height，内联元素不行！内联元素想要设定位置，需要通过包裹它的元素设置，或将其设为 display: block。Overflow 属性有 4 个值：

- visible（默认，内容不被修剪，显示在元素框之外）

- hidden（内容被修剪，超出的部分不可见）

- scroll（内容会被修剪，但是提供滚动条以查看）

- auto：由浏览器定夺。如果修建，会提供滚动条。

Padding 表示内边距。可以通过 padding 属性一次设置 4 个边：

padding: 10px; padding: 10px 0.25em 2ex 20%;

按照 top-right-bottom-left，顺时针方向。也可以分开设置，padding-top、padding-right，等等。

Border 表示边框。默认为 0。可以通过 border 属性设置宽度、风格、颜色。

或者，border-top 可以一次设置某边的宽度、风格、颜色；

再或者，border-width 可以单独设置，应用到四边，顺序按照顺时针，同上；

还可以, border-top-width, 单独设置某边的三种属性。

注意, border-width 单独使用不起作用, 必须先使用 border-style 设置边框。

属性 Background-clip, 背景裁剪。包含 border-box、padding-box、content-box。能够裁剪元素中的内容, 按照上述盒模型的概念裁剪。

Margin 表示外边距。默认为 0, 在 CSS 布局中**推开**其他盒子。可以通过 margin 一次性设置宽度、风格、颜色, 或 margin-top 四边分别设置。外边距默认是透明的。

注意, 相邻的块级元素, 其上 margin 和下 margin 有时会合并成一个 margin, 大小取两者之中的最大者, 称为“外边距重叠”。比如, 两个元素之间的距离有多远? 是 $20+20=40\text{px}$ 吗? 由于存在外边距重叠现象, 实际是 $\max(20,20)=20\text{px}$ 。

(1)在参与折叠的 margin 都是正数的情况下, 取其中较大的值为最终 margin;

(2)在参与折叠的 margin 都是负值的时候, 取其中绝对值较大的, 然后从零开始, 负向位移;

(3)在参与折叠的 margin 有正值有负值的时候, 要从所有负值中选出绝对值最大的, 所有正值中选择绝对值最大的, 二者相加, 如 $100\text{px}+(-50\text{px})=50\text{px}$ 。

对于块级元素, 如果其 margin-left 和 margin-right 都设定为 auto, 则可将块级元素居中。

对于内联元素的盒模型, 设置宽度和高度未生效, 外边距、内边距和边框生效, 且 margin-top 和 margin-bottom 被忽略。内联元素仍然会处于内联的行中, 和其它元素关系保持不变。如果使用 display: inline-block, 不会跳转到新行, 显式添加 width 和 height 只会让它变大。

对于<body>元素, 默认带有一定的 margin 值。

3.5 布局

正常文档流: 一般的 HTML 元素在页面出现的先后顺序。

脱离正常文档流: 用 CSS 控制元素的显示位置和文档代码顺序不一致。可以通过 float、position、display 属性控制。

- float, 浮动。允许元素浮动到另外一个元素的左侧或右侧, 而不是默认的一个堆叠另一个, 包含 left (将元素浮动到左侧)、right、none、inherit。主要用途是布置出多个列并且浮动文字以环绕图片。注: 如果元素 1 和元素 2 同时为浮动元素, 外边距就会生效!

一旦对一个元素进行了浮动, 所有接下来的元素都会环绕它。如果不想在某个元素受到其之前的浮动元素影响时, 可以为其添加 clear 属性清除浮动, 接下来的部分会以正常流方式绘制在其后。clear 属性包含 left (清除左浮动)、right、both (左右一起清除)。

可以使用 `float` 来创建双栏布局。一个 `float: left`，一个 `float: right`。为使它们能够在同一行放下来，注意水平间距总宽度（`width` 的和）不要大于 100%!

- position，定位。包含四种：`static`（默认值）、`relative`、`absolute`、`fixed`。配合 `top`、`bottom`、`left`、`right` 这四个属性来控制元素位置。

当 `position` 属性为 ***fixed*** 时，视野中的元素位置不会再改变；

当 `position` 属性为 ***relative*** 时，其位置是相对于它的原始位置计算而来的（一种偏移）；

当 `position` 属性为 ***absolute*** 时，绝对定位能够很精确地把元素移动到任意位置。默认情况下，绝对定位的元素的位置相对于最近的已定位父元素；如果元素没有已定位的父元素，那么它的位置相对于<html>。相对定位的位置是相对原始位置而言。一旦变为绝对定位元素，该元素就脱离正常文档流，其原本占据的空间也会被移除。

当 `position` 属性为 ***static*** 时，一切照常。

- display。

当 `display` 设置为 `none` 时，元素会被忽略（整个不存在）；如果是 `visibility: hidden`，只是元素隐身了，但它仍然占据那个位置。

`display` 为 `inline`，内联元素。

`display` 为 `inline-block`。可以设置父元素 `font-size:0`；来去除 `inline-block` 元素的间距。

`display` 为 `flex`，实现 Flexbox 布局，用于设计横向或纵向，即排在一排，没有间隔。

`display` 为 `grid`，实现 Grid 布局，用于同时在两个维度上把元素按行和列排列整齐。还可以使用 `grid-template-rows` 和 `grid-template-columns` 两个属性定义行和列。

4 JavaScript

JavaScript（脚本语言，使用解释器）的核心语法只包括两个部分：基本的语法构造（比如操作符、控制结构、语句）和标准库（就是一系列具有各种功能的对象比如 `Array`、`Date`、`Math` 等）。

除此之外，各种宿主环境提供额外的 API（即只能在该环境使用的接口），以便 JavaScript 调用。

JavaScript 在 HTML 中的引用方式：

（1）在 `<head>` 或 `<body>` 之间编写 JS：

```
<script type="text/javascript">
```

```
// 这里编写 js 程序
```

</script>

(2) 使用 script 标签来引用。script 标签可以放在 head 标签内，也可以放在 body 标签中：<script src="scripts/main.js"></script>

如果脚本文件使用了非英语字符，应该注明字符的编码，所加载的脚本必须是纯 JavaScript 代码，不能有 HTML 代码和<script>标签：

<script charset="utf-8" src="https://www.example.com/script.js"></script>

为了防止攻击者篡改外部脚本，script 标签允许设置一个 integrity 属性，写入该外部脚本的 Hash 签名，用来验证脚本的一致性：

<script src="/assets/application.js" integrity="sha256-xxxxxxx"></script>

(3) 可以在元素事件中引入，即在元素的某一个属性中直接编写 JavaScript 程序或调用 JavaScript 函数，这个属性指的是元素的“事件属性”如 input 标签的 onclick。

4.1 浏览器加载 js 的过程

浏览器加载 JavaScript 脚本，主要通过<script>元素完成，正常的流程：

(1) 浏览器一边下载 html 网页，一边开始解析；

(2) 解析过程中，浏览器一旦发现<script>元素，就暂停解析，把网页渲染控制权交给 JavaScript 引擎；

(3) 如果<script>元素引用了外部脚本，就下载该脚本再执行，否则就直接执行代码；

(4) JavaScript 引擎执行完毕，控制权交还渲染引擎，继续往下解析网页。

如果外部脚本加载时间很长（一直无法完成下载），那么浏览器就会一直等待脚本下载完成，这被称为“**阻塞效应**”。为了避免这种情况，较好的做法是将<script>标签都放在页面底部，而不是头部。

如果某些脚本代码非常重要，一定要放在页面头部的话，最好直接将代码写入页面。脚本文件都放在网页尾部加载，还有一个好处：在 DOM 结构生成之前就调用 DOM 节点，JavaScript 会报错；如果脚本都在网页尾部加载，就不存在这个问题，因为这时 DOM 肯定已经生成了。

如果**有多个 script 标签**，<script src="a.js"></script><script src="b.js"></script> 浏览器会同时并行下载 a.js 和 b.js，但是，执行时会保证先执行 a.js，然后再执行 b.js，即使后者先下载完成。

4.2 Js 的编译过程

词法分析 -> 语法分析 -> 使用翻译器，将代码转换为字节码 -> 使用解释器，将字节码转为机器码

现代浏览器采用“即时编译”（Just In Time compiler，缩写 JIT）。在使用 JIT 编译器的语言中，源代码会首先被解释器解释执行，而当某些部分的代码被频繁执行时，JIT 编译器会将这些部分的代码编译成机器码，从而提高执行速度。

字节码不能直接运行，而是运行在一个虚拟机（Virtual Machine）之上，一般也把虚拟机称为 JavaScript 引擎。浏览器的“开发者工具”里面的“控制台”，就是运行 JavaScript 代码的理想环境。

4.3 基本语法

4.3.1 语句和表达式

JavaScript 程序的执行单位为行。

语句（statement）指为了完成任务而进行的操作，语句以分号结尾，一个分号就表示一个语句结束；多个语句可以写在一行内。一般无返回值。

表达式（expression）指为了得到返回值的计算式。总是有返回值。`var a = 1+3;`

4.3.2 变量

变量，有声明，有赋值，区分大小写。JavaScript 是一种动态类型语言，变量的类型没有限制，变量可以随时更改类型。`var a = 1; a = 'hello';`（允许）

`var`、`let` 和 `const` 都是 JavaScript 中用来声明变量的关键字。

var 与 let 的区别——作用域区别：

-- `var` 声明的变量的作用域只能是全局或者整个函数块的；

-- `let` 声明的变量的作用域则是它当前所处代码块（可能只在 if 语句内）；

-- `var` 允许在同一作用域中重复声明，而 `let` 不允许，否则将抛出异常

let 与 const 异同：`const` 具有上面提到的 `let` 的特性，唯一区别就在于 `const` 声明的是一个只读变量，声明之后不允许改变其值。因此，`const` 一旦声明必须初始化。

JavaScript 的数据类型分为两类：原始值类型和对象（Object 类型）。这一点是仿照 java 的。

`const` 保证变量指向的内存地址所保存的数据不允许改动。所以，对于对象类型的值，`const` 只能保证指针是不可修改的，至于指针指向的数据结构是无法保证其不能被修改的。比如，`const obj={value: 1}`，禁止 `obj={}`，但允许 `obj.value=2`。

变量提升：所有的变量声明语句，都会被提升到代码的头部。比如：

`console.log(a); var a=1; ——> var a; console.log(a); a=1;`

这种机制不会报错，但是是错误的，因为 `a` 会输出 `undefined`。

4.3.3 标识符

标识符指用来识别各种值的合法名称。如变量名、函数名等。对大小写敏感。
规则：第一个字符可以是任意 Unicode 字母(包括英文字母和其他语言的字母)，以及美元符号 (\$) 和下划线 (_)；第二个字符及以后，除了上述三种，还可以用数字 0-9。

因此，标识符不能有加号、星号、减号、连线符等。还不能是保留字。

4.3.4 注释

单行注释：//

多行注释：/* */

注意：HTML 注释：<!-- -->，CSS 注释：/* */

4.3.5 条件语句

If(...) {statement;} else if(...) {statement;} else {statement;}，基本和 C 一样。

注意：严格相等运算符 (===) 和相等运算符 (==) 的区别：

- 前者比较是否为 “同一个值”；
- 后者比较两个值是否相等。

如果两个值不是同一类型，严格相等运算符 (===) 直接返回 false，而相等运算符 (==) 会将它们转换成同一个类型，再用严格相等运算符进行比较。

```
switch(...) {  
  case "apple":  
    break;  
  case "banana":  
    break;  
  case 1:  
    break;  
  default:  
    //...  
}
```

基本和 C 一样。注意 break 语句不能省略。上面在比较时，使用的是严格相等运算符 (===)，因此不会发生类型转换。

三元运算符： var a = (n === 1) ? true : false; 对则 “true”，错则 “false”。

4.3.6 循环

```
while (...) { ... }
```

```
for(...; ...; ...) {}
```

```
do{ ... }while (...);
```

while、for、do-while，和 C 一样。do-while 结尾有一个分号。break、continue 也是一样的，还可以使用标签，配合 break/continue [label-name]使用：

```
top: （或者其他 label-name）
```

```
语句
```

4.4 数据类型

一共六种：

（1）数值（number）：整数和小数（比如 1 和 3.14）

（2）字符串（string）：文本（比如 Hello World）

（3）布尔值（boolean）：表示真伪的两个特殊值，即 true（真）和 false（假）

（4）undefined：表示“未定义”或不存在，即由于目前没有定义，所以此处暂时没有任何值

（5）null：表示空值，即此处的值为空

（6）对象（object）：各种值组成的集合

数值、字符串、布尔值合称为原子类型（primitive type）；对象则称为复合类型（complex type）；undefined 和 null 可以看成两个特殊值。

对象又可以分成三个子类型：狭义的对象、数组、函数。

typeof 运算符可以返回一个值的数据类型。由于历史兼容问题，null 的类型被视为对象，即 object。

4.4.1 null & undefined

null 与 undefined 都可以表示“没有”，在 if 语句中它们都会被自动转为 false，相等运算符（==）甚至直接报告两者相等。

null 表示“此处不应该有值”，转为数值时为 **0**；undefined 表示“缺少值”，转为数值时为 NaN，就是此处应该有一个值，但是还没有定义。

4.4.2 布尔值

值有 true 和 false。运算符有：!, ===, !==, ==, !=, >, >=, <, <=。

如果 js 预期某个位置应该是布尔值，会将该位置上现有的值自动转为布尔值，除了以下 6 个被转为 false，其余都是 true：

undefined、null、false、0、NaN、""或”空字符串

也就是说，空数组[]、空对象{}都是 true。

4.4.3 数值

JavaScript 中只有一种数字类型：基于 IEEE 754 标准的双精度 64 位二进制格式的值。所以，整数也是 64 位浮点数形式存储。

除了浮点数外，js 中还有一些带符号的值：

+Infinity, -Infinity 和 NaN

绝对值小于 2 的 53 次方的整数，都可以精确表示。

由于浮点数不是精确的值，所以涉及小数的比较和运算要特别小心。

$(0.3-0.2) == (0.2-0.1)$ ，结果是 *false*。

数值的表示方法，有二/八/十/十六进制（内部会自动将二/八/十六进制转为十进制）或科学计数法（123e-3）。

遇见以下两种情况，js 会自动将数值转为科学计数法表示：

小数点前的数字多于 21 位；小数点后的零多于 5 个

特殊数值：零、正零、负零。三者等价。唯一区别是在作分母时， $(1 / +0)$ 、 $(1 / -0)$ 会出现 Infinity 和 -Infinity。

以及特殊数值 NaN, Not a Number。比如计算 $5 - 'x'$ ，会自动将字符串 x 转为数值。但由于 x 不是数值，所以得到结果 NaN。数学函数发生错误时，也会返回 NaN（ $0 / 0 = \text{NaN}$ ）。注意！NaN 不等于任何值，包括它本身（ $\text{NaN} === \text{NaN}$ ，得到 false）。因此，NaN 与任何数（包括它自己）的运算，得到的都是 NaN。

还有 Infinity，无穷。Infinity 大于一切数值（除了 NaN），-Infinity 小于一切数值（除了 NaN），Infinity 与 NaN 比较，总是返回 false。其四则运算按照常理判断即可。

与数值有关的全局方法：

parseInt 方法用于将字符串转为整数。可以接受第二个参数（2 到 36 之间），表示被解析值的进制，返回该值对应的十进制数。如果字符串的第一个字符不能转化为数字，返回 NaN。字符串头部空格会被自动去除。如果参数不是字符串，则会先转为字符串再转换。字符串转为整数的时候，是一个个字符依次转换，如果遇到不能转为数字的字符，就不再进行下去，返回已经转好的部分。十六进制数、科学计数法，都可以转换。

isNaN 方法可以用来判断一个值是否为 NaN，isNaN 只对数值有效，如果传入其他值，会被先转成数值。对于空数组 [] 和只有一个成员的数组 [100]，isNaN 返回 false，原因是这些数组能被 Number 函数转成数值。

isFinite 方法返回一个布尔值，表示某个值是否为正常的数值。除了 Infinity、-Infinity、NaN 和 undefined 这几个值会返回 false，isFinite 对于其他的数值都会返回 true。

4.4.4 字符串

多行字符串，使用“+”或反斜杠来实现。

字符串可以被视作字符数组，因此可以使用数组的方括号运算符，用来返回某个位置的字符。

如果方括号中的数字超过字符串的长度，或者方括号中根本不是数字，则返回 `undefined`。字符串内部的单个字符无法改变和增删。操作会默默地失败，不会返回错误信息。

`length` 属性返回字符串的长度，该属性也无法改变，也不会报错。（`s.length`）

js 允许在程序中直接将字符写成 `\uxxxx` 的形式。`s = '\u11A9'`，`var fu006F\u006F = 'abc'`；访问 `foo` 能够得到 `abc`。

js 提供两个 Base64 相关的方法：

`btoa()`：任意值转 Base64；`atob()`：Base64 转原始值。

但是，非 ASCII 码进行操作会报错，比如中文。

4.4.5 对象

对象就是一组“键值对”（`key-value`）的集合，是一种无序的复合数据集合。

```
var obj = { foo: 'hello', bar: 'world' };;
```

对象的所有键名都是字符串，里面加不加引号都可以。如果键名是数值，会被自动转为字符串。如果键名不符合标识符的条件，且不是数字，则必须加上引号，否则会报错。

对象的键名又称为“属性”（`property`），对象的“键值”可以是任何数据类型。如果一个属性的值为函数，通常把这个属性称为“方法”，它可以像函数那样调用。如果属性的值还是对象，就形成了链式引用。

对象的属性之间用逗号分隔，最后一个属性后面可以加逗号，也可以不加。属性可以动态创建，不必在对象声明时就指定。

如果不同的变量名指向同一个对象，那么它们都是这个对象的引用，也就是说指向同一个内存地址。取消某一个变量对于原对象的引用，不会影响到另一个变量对原对象的引用。只有对象叫“引用”，原子类型都是“拷贝”。

读取对象的属性，有两种方法：点运算符、方括号运算符。如果使用方括号运算符，键名必须放在引号里面，否则会被当作变量处理。方括号内部可以放表达式。

```
var obj = { foo: 1, bar: 2};
```

`obj.foo` 和 `obj['foo']` 会得到 1；

`obj[foo]`，`foo` 会被系统当作变量，指向字符串 `bar`，所以得到结果 2。

数字键名可以不加引号，但数值键名不能使用点运算符。

由于属性可以在运行时动态添加，自然可以删除，使用 `delete` 命令。

```
delete obj.foo
```

删除一个不存在的属性，`delete` 不报错，而且返回 `true`！`delete` 命令只能删除对象本身的属性，无法删除继承的属性。

`in` 运算符用于检查对象是否包含某个属性/键名。

```
'foo' in obj
```

可以使用对象的 `hasOwnProperty` 方法判断是否为对象自身的属性：

```
obj.hasOwnProperty('toString') // 返回 false
```

属性的遍历。`for...in` 循环用来遍历一个对象的全部属性：

```
for (var i in obj) { statements... }
```

它会遍历所有可遍历属性，不可遍历的会自动跳过。

4.4.6 对象属性——函数

三种函数方法：

(1) 函数的声明：

```
function con_print(s) { ... }
```

(2) 函数表达式（采用变量赋值的方法）：

```
var print = function(s) { ... };
```

`function` 命令后面不带有函数名时，为匿名函数。如果加上函数名，该函数名只在函数体内部有效，在函数体外部无效。

函数声明结尾不用分号；而函数表达式需要。

(3) `Function` 构造函数：

可以传递任意数量的参数给 `Function` 构造函数，只有最后一个参数会被当做函数体；如果只有一个参数，该参数就是函数体。可以不使用 `new` 命令，返回结果完全一样。

```
var add = new Function( 'x', 'y', 'return x + y' );
```

其实就是个求和函数。

注意，如果同一个函数被多次声明，后面的声明就会覆盖前面的声明。由于函数名提升现象，前一次声明在任何时候都是无效的。

`return` 语句不是必需的，如果没有的话，该函数就不返回任何值，或者说返回 `undefined`。函数可以调用自身。

“函数为第一等公民”：凡是可以使用值的地方，就能使用函数。

```
function a(op) { return op; }      a(add)(1,1) // 返回 2
```

变量提升 (hoisting)：函数声明和变量声明都会跑到代码头部。如果是赋值语句定义函数，就不存在变量提升。如果同时采用 function 声明和赋值语句定义同一个函数，最后总是采用赋值语句的定义，因为变量提升后，先声明、后赋值，赋值的定义覆盖了之前的声明。

函数的 name 属性返回函数的名字：f1.name // “f1”

函数如果有名字，就是函数的名字；否则（匿名函数），就是变量的名字。

函数也有 length 属性。会返回函数定义之中的参数个数。

函数 toString 方法返回一个字符串，内容是函数的源码，包括函数内的注释。

关于作用域：Js 只有两种作用域：全局作用域(全局变量)、函数作用域(局部变量)。函数内部定义的局部变量，能够覆盖全局变量。所有作用域内，都有变量提升现象。函数的作用域与变量一样，就是其声明时所在的作用域，与其运行时所在的作用域无关。

函数参数。参数不是必须的，无论提供多少参数，JavaScript 都不会报错。省略的参数值变为 undefined。没办法只省略靠前的参数，而保留靠后的参数，除了显式传入 undefined 来跳过。

如果有同名的参数，则取最后出现的那个值，即使后面的没有值或被省略（变成 undefined）。

参数传递有两种：传值传递（原子类型，不影响原值）、传址传递（复合类型，影响原值）。但是，如果函数内部修改的，不是参数对象的某个属性，而是把参数对象整个替换成另一个值，这时不会影响到原始值。因为重新对对象赋值，导致其指向另一个地址，保存在原地址上的值（也就是传入的参数）不受影响。

arguments 对象只在函数体内部才可以使用，包含了函数运行时的所有参数。

`f(1, 2, 3, 4, 5)` // 若调用 arguments.length，此时为 5

正常模式下，arguments 对象可以在运行时修改：

`arguments[0] = 3;` // 在 function 中，会改变参数的值

严格模式下 arguments 对象是一个只读对象，修改它是无效的，但不会报错：

`'use strict';` // 使用严格模式

`arguments[0] = 3;` // 无效

4.4.7 对象的一种——数组

数组 (array) 是按次序排列的一组值。

`var arr = [];`

`arr[0] = 1;`

`arr[1] = function(){...};`

数组可以先定义后赋值。任何类型的数据，都可以放入数组。

本质上，数组属于一种特殊的对象。`typeof` 运算符会返回数组的类型是 `object`。数组的 `length` 属性返回数组的成员数量，可以通过修改 `length` 属性值随时增减数组的成员(`arr.length`)。`length` 设置大于当前个数，多出来的位置都是 `undefined`；减少了就直接截断。

数组可以添加属性，但如果该键名非数字，就不会影响 `length` 的值。`length` 属性的值就是等于最大的数字键加 1。当使用字符串作为键添加元素时，它们实际上并不算作数组的元素，而是作为对象的属性添加到数组对象中。数组的长度只统计数组的数值索引元素的个数，而不包括非数值索引的属性。

`in` 运算符适用于对象，也适用于数组，用于检查某个键名是否存在。

`for...in` 循环，不仅会遍历数组所有数字键，还会遍历非数字键，不推荐使用 `for...in` 遍历数组。可以考虑使用 `for` 或 `while`。

当数组的某个位置是空元素，即两个逗号之间没有任何值 (`[1,,1,]`)，我们称该数组存在空位 (`hole`)，数组的空位不影响 `length` 属性。最后一个元素后面有逗号，不会产生空位。空位是可以读取的，返回 `undefined`。使用 `delete` 命令删除一个数组成员，会形成空位，并且不会影响 `length` 属性。比如：

`a = [1, 2,, 4]`，`delete a[0]`后，剩下`[, 2,, 4]`

数组的某个位置是空位，与某个位置是 `undefined`，是不一样的；空位就是数组没有这个元素，所以不会被遍历到，而 `undefined` 则表示数组有这个元素，值是 `undefined`，所以遍历不会跳过。

4.5 运算符

4.5.1 加法+

(1) 数字+布尔值：布尔值都会自动转成数值，然后再相加；

(2) 字符串+字符串：加法运算符会变成连接运算符；

(3) 非字符串+字符串：非字符串会转成字符串，再连接在一起。如果运算符是对象，必须先转成原子类型的值，然后再相加。对象转成原子类型的值计算：

`obj.valueOf().toString()` // 得到“[object Object]”

自增(`++`)和自减(`--`)，会修改原始变量。其它没提到的不会。

4.5.2 不等符号

对于不等符号 (`<`, `>`, ...) 比较规则：先看两个运算符是否都是字符串，如果是，就按照字典顺序比较；否则，将两个运算符都转成数值，再比较数值的大小。

如果运算符是对象，会转为原子类型的值，再进行比较。对象转换成原子类型的值，算法是先调用 `valueOf` 方法；如果返回的还是对象，再接着调用 `toString`

方法。

4.5.3 严格相等 ===

- (1) 如果两个值的类型不同，直接返回 `false`;
- (2) 同一类型的原始类型比较时，值相同就返回 `true`，值不同就返回 `false`;
- (3) 两个复合类型（对象、数组、函数），比较它们是否指向同一个地址。比如，运算符两边的空对象、空数组、空函数的值，都存放在不同的内存地址，结果是 `false`;
- (4) 值得一提的是，对于两个对象的比较，严格相等运算符比较的是地址，而大于或小于运算符比较的是值;
- (5) `null`、`undefined` 和自身严格相等。

对于严格不相等符号 (`!==`)，计算严格相等的结果，再取反。

4.5.4 相等运算符 ==

比较相同类型的数据时，与严格相等运算符完全一样。

比较不同类型的数据时，先将数据进行类型转换，然后再用严格相等运算符比较。

(1) 原始类型的值会转换成数值再进行比较。一个例子：`'true'==true`，转为 `Number('true')==Number(true)`，最后 `NaN===1`，结果是 `false`;

(2) 对象（这里指广义的对象，包括数组和函数）与原始类型的值比较时，对象转换成原始类型的值，再进行比较;

(3) `undefined` 和 `null` 与其他类型的值比较时，结果都为 `false`，它们互相比时（包括与自己）结果为 `true`;

不相等运算符 (`!=`) 即求相等运算结果，再取反。

4.5.5 布尔运算符

取反!

且运算符 `&&`: 运算符可以多个连用，这时返回第一个布尔值为 `false` 的表达式的值。如果所有表达式的布尔值都为 `true`，则返回最后一个表达式的值。

或运算符 `||`

三元条件运算符 `?:`

4.5.6 二进制位运算符

或(`|`)，与(`&`)，否(`~`)，异或(`^`)，左移(`<<`)，右移(`>>`)，无符号右移(`>>>`)

4.6 类型转换

4.6.1 强制类型转换

Number(): 强制转换成数字。要么“是”数字，要么“不是”数字 (NaN)。只要有一个字符无法转成数值，整个字符串就会被转为 NaN。

参数是对象，返回 NaN，除非是包含 0 个或者单个数值的数组。

String(): 强制转换成字符串。数值、布尔值、undefined、null，都变成自己的原值加引号，作字符串。“123”、“true”、“undefined”、“null”。

参数如果是狭义对象，返回一个类型字符串；

参数如果是函数，返回函数.toString 值；

参数如果是数组，返回该数组的字符串形式。比如 String([1,2,3]) -> “1,2,3”

Boolean(): 强制转换成布尔值。

4.6.2 自动类型转换

自动转换是以强制转换为基础的。自动转换的规则是：预期什么类型的值，就调用该类型的转换函数。

4.7 标准库

4.7.1 全局函数

(1) eval()

参数是一个字符串，则将这个字符串当作语句执行。如果 eval 的参数不是字符串，那么会原样返回。eval 的本质是在当前作用域之中，注入代码。

(2) isFinite()

如果参数是 NaN, +Infinity 或者 -Infinity, 会返回 false, 其他视情况返回 true。

(3) isNaN()

确定一个值是不是 NaN。

(4) parseInt()

解析一个字符串参数，并返回一个指定基数的整数。开头和结尾的空白符允许存在，会被忽略。

(5) parseFloat()

解析一个字符串参数并返回一个浮点数。开头和结尾的空白符允许存在，会被忽略。基本规则同 parseInt

4.7.2 URI 编解码函数

(1) escape()

可对字符串进行编码。方法不会对 ASCII 字母和数字进行编码，也不会对下面这些 ASCII 标点符号进行编码：* @ - _ + . / 。其他所有的字符都会被转义序列替换。

4.7.3 字符串对象 string 相关方法

match 方法

用于确定原字符串是否匹配某个子字符串。没有匹配返回空数组。

`"string".match('str')`

search 方法：

用法基本等同于 match，但是返回值为匹配的第一个位置，如果没有找到匹配，则返回 -1；

replace 方法：

用于替换匹配的子字符串，一般情况下只替换第一个匹配（除非使用带有 g 修饰符的正则表达式）；

charAt 方法：

返回指定位置的字符，参数是从 0 开始编号的位置。如果参数为负数，或大于等于字符串的长度，返回空字符串；

concat 方法：

用于连接两个字符串，返回一个新字符串，不改变原字符串。如果参数不是字符串，concat 方法会将其先转为字符串，然后再连接；

slice 方法：

用于从原字符串取出子字符串并返回，不改变原字符串，第一个参数是子字符串的开始位置，第二个参数是子字符串的结束位置（不含该位置）；

indexOf 方法：

用于确定一个字符串在另一个字符串中第一次出现的位置，返回结果是匹配开始的位置。如果返回 -1，就表示不匹配。lastIndexOf 从尾部开始匹配，indexOf 则是从头部开始匹配；

trim 方法：

用于去除字符串两端的空格（space、\t、\n、\r），返回一个新字符串，不改变原字符串；

toLowerCase/toUpperCase 方法：

和名字一样。不改变原字符串；

split 方法：

按照给定规则分割字符串，返回一个由分割出来的子字符串组成的数组。详细规则如下：

- 1) 如果分割规则为空字符串, 则返回数组的成员是原字符串的每一个字符;
- 2) 如果省略参数, 则返回数组的唯一成员就是原字符串;
- 3) 如果满足分割规则的两个部分紧邻着或位于开头结尾, 则返回数组之中会有一个空字符串;
- 4) 可以接受第二个参数, 限定返回数组的最大成员数。

localeCompare 方法:

用于比较两个字符串, 返回一个整数, 如果小于 0, 表示第一个字符串小于第二个字符串; 如果等于 0, 表示两者相等; 如果大于 0, 表示第一个字符串大于第二个字符串。该方法的最大特点, 就是会考虑自然语言的顺序 (不是按 unicode/ascii 计算);

4.7.4 日期对象 Date

Date 对象可以作为普通函数直接调用, 返回一个代表当前时间的字符串。

Date(); // 给出当前时间

Date.now 方法:

返回当前时间距离时间零点 (1970 年 1 月 1 日 00:00:00 UTC) 的毫秒数;

Date.parse 方法:

用来解析日期字符串, 返回该时间距离时间零点 (1970 年 1 月 1 日 00:00:00) 的毫秒数;

Date.UTC 方法:

接受年、月、日等变量作为参数, 返回该时间距离时间零点 (1970 年 1 月 1 日 00:00:00 UTC) 的毫秒数;

Date.UTC(year, month[, date[, hrs[, min[, sec[, ms]]]])

Date 还可以当作构造函数使用。对它使用 new 命令, 会返回一个 Date 对象的实例。如果不加参数, 实例代表的就是当前时间; 可以接受多种格式的参数, 返回一个该参数对应的时间实例。

new Date() // 返回 Date 对象, 和 Date() 不同, 不是字符串!

new Date-1 // 会返回毫秒数

日期的运算:

- 类型自动转换时, Date 实例如果转为数值, 则等于对应的毫秒数; 如果转为字符串, 则等于对应的日期字符串;

- 两个日期实例对象进行减法运算时, 返回的是它们间隔的毫秒数; 进行加法运算时, 返回的是两个字符串连接而成的新字符串。

Date 的实例对象常见的方法可以分为以下三类:

to 类: toString, toUTCString, toLocaleString

get 类：获取 Date 对象的日期和时间。**getMonth()：**返回月份（0 表示 1 月，11 表示 12 月）；

set 类：设置 Date 对象的日期和时间。凡是涉及到设置月份，都是从 0 开始算的，即 0 是 1 月，11 是 12 月。Set 类方法的参数都会自动折算。以 setDate 为例，如果参数超过当月的最大天数，则向下一个月顺延；如果参数是负数，表示从上个月最后一天开始减去的天数。

4.7.5 数组对象 Array

构造函数：new Array()；参数可以是单个整数参数 n，创建长为 n 的数组；可以是单个非数值/多个参数，返回由参数创建的数组。

new Array(4); // 创建长为 4 的数组

new Array('abc'); // 创建元素为 abc 的数组

Array.isArray 方法：

返回一个布尔值，表示参数是否为数组，它可以弥补 typeof 运算符的不足（typeof 运算符只能显示数组的类型是 Object）。

push(), pop()方法：

push 用于在数组的末端添加一个或多个元素，并返回添加新元素后的数组长度；pop 用于删除数组的最后一个元素，并返回该元素，对空数组使用 pop 方法，不会报错，而是返回 undefined。两种方法会改变原数组！

shift(), unshift()方法：

shift()方法用于删除数组的第一个元素，并返回该元素。unshift()方法用于在数组的第一个位置添加元素，并返回添加新元素后的数组长度。

join()方法：

以指定参数作为分隔符，将所有数组成员连接为一个字符串返回。如果不提供参数，默认用逗号分隔。如果数组成员是 undefined 或 null 或空位，会被转成空字符串。

concat 方法：

用于多个数组的合并。它将新数组的成员，添加到原数组成员的后部，然后返回一个新数组，原数组不变。

["hello"].concat(['world'], ['!']) // 得到三个元素的数组

[1, 2, 3].concat(4, 5, 6) // 1,2,3,4,5,6

reverse 方法：

用于颠倒排列数组元素，返回改变后的数组。该方法将改变原数组。

slice 方法：

用于提取目标数组的一部分，返回一个新数组，原数组不变。以下是 slice 的

规则：slice(start, end)

- 如果省略所有参数，返回原数组的拷贝
- 如果省略第二个参数，则一直返回到原数组的最后一个成员
- 如果 slice 方法的参数是负数，则表示倒数的位置
- 如果第一个参数大于等于数组长度，或者第二个参数小于第一个参数，则

返回空数组

splice(start, count, addElement1, addElement2, ...):

用于删除原数组的一部分成员，并可以在删除的位置添加新的数组成员，返回值是被删除的元素。该方法会改变原数组。第一个参数是删除的起始位置（从 0 开始），第二个参数是被删除的元素个数。后面的参数表示要被插入的新元素。

- 起始位置是负数表示从倒数位置开始删除
- 第二个参数设为 0 表示单纯地插入元素
- 只提供第一个参数表示删除到结尾。

sort 方法:

对数组成员进行排序，默认是按照字典顺序排序。数值会被先转成字符串，再按照字典顺序进行比较。原数组将被改变。

map 方法:

将数组的所有成员依次传入参数函数，然后把每一次的执行结果组成一个新数组返回，原数组没有变化。函数调用时，map 方法向它传入三个参数：当前成员、当前位置和数组本身。如果数组有空位，map 方法的 return 函数在这个位置不会执行，会跳过数组的空位，但不会跳过 undefined 和 null。

forEach:

的用法与 map 方法一致，参数是一个函数，该函数同样接受三个参数：当前值、当前位置、整个数组。方法会跳过数组的空位，但不会跳过 undefined 和 null。

filter 方法:

用于过滤数组成员，满足条件的成员组成一个新数组返回，该方法不会改变原数组。参数是一个函数，函数有三个参数：当前成员，当前位置和整个数组。所有数组成员依次执行该函数，返回结果为 true 的成员组成一个新数组返回。

some(), every():

返回一个布尔值，表示判断数组成员是否符合某种条件。some 方法判断是否存在符合条件的成员，every 方法判断是否所有成员都符合条件。

reduce(), reduceRight():

方法依次处理数组的每个成员，最终累计为一个值（原数组转变而来）。reduce 是从左到右处理，reduceRight 则是从右到左。参数是一个函数。

`indexOf()`, `lastIndexOf()`:

返回元素在数组中第一次/最后一次出现的位置, 如果没有出现则返回-1。方法都可以接受第二个参数, 表示搜索的开始位置。

4.7.6 Math 对象

该对象不是构造函数, 不能生成实例, 所有的属性和方法都必须在 `Math` 对象上调用。

`Math.random()`:

返回[0,1)之间的一个伪随机数。可以用 `random` 构造随机数生成, 随机整数生成, 随机字符串生成。

4.8 JavaScript 与 DOM

DOM (Document Object Model, 文档对象模型)

要实现页面的动态交互和效果, 操作 HTML 是核心, 操作 HTML 其实就是对 DOM 的操作。**DOM 提供了用程序来动态控制 HTML 的接口**, 处在为 JavaScript 赋予 HTML 具备动态交互效果的能力的核心地位上。

在 HTML DOM 中, 所有事物都是节点。DOM 节点的主要类型有七种:

- 元素节点: `Element`。html 元素, 比如 `head`、`body`
- 属性节点: `Attr`。HTML 元素的属性如 `id`、`class`、`href` 等都属于 `Attr`
- 文本节点: `Text`。文本节点是只包含文本内容的节点, 文本节点可以只包含空格
- 注释节点: `Comment`。注释内容。
- 文档节点: `Document`。文档节点是文档树的根节点。
- 文档类型节点: `DocumentType`。比如声明文档类型时 `<!DOCTYPE html>`
- 文档片段节点: `DocumentFragment`。是轻量级的或最小的 `Document` 对象, 它表示文档的一部分或者是一段, 不属于文档树。

DOM 提供操作接口, 用来获取这三种关系的节点:

- 父节点关系 (`parentNode`): 直接的那个上级节点
- 子节点关系 (`childNodes`): 直接的下级节点
- 兄弟节点关系 (`sibling`): 拥有同一个父节点的节点

4.8.1 节点 Node——DOM 操作的基础

所有 DOM 节点对象都继承了 `Node` 接口, 拥有一些共同的属性和方法。

`nodeType` 属性:

返回一个整数值, 表示节点类型, `Node` 对象定义了几个常量, 对应这些类型值。常用于确定节点类型。

nodeName 属性:

返回节点的名称。

nodeValue 属性:

返回一个字符串，表示当前节点本身的文本值，该属性可读写。只有文本节点（text）、注释节点（comment）和属性节点（attr）有文本值，nodeValue 可以返回结果，其他类型的节点一律返回 null。

nextSibling 属性:

返回紧跟在当前节点后面的第一个同级节点，如果当前节点后面没有同级节点，则返回 null。该属性还包括文本节点和注释节点，如果当前节点后面有空格，该属性会返回一个文本节点，内容为空格。

previousSibling 属性:

返回当前节点前面的、距离最近的一个同级节点。如果当前节点前面没有同级节点，则返回 null。该属性还包括文本节点和注释节点。因此如果当前节点前面有空格，该属性会返回一个文本节点，内容为空格。

parentNode 属性:

返回当前节点的父节点，对于一个节点来说，它的父节点只可能是三种类型:

- 元素节点（element）
- 文档节点（document）
- 文档片段节点（documentfragment）

parentElement 属性:

返回当前节点的父元素节点，如果当前节点没有父节点，或者父节点类型不是元素节点，则返回 null。

firstChild/lastChild 属性:

返回当前节点的第一个/最后一个子节点，可能是元素节点、文本节点或注释节点，如果当前节点没有子节点，则返回 null。

childNodes 属性:

返回一个类似数组的对象集合 NodeList，成员包括当前节点的所有子节点。

isConnected 属性:

返回一个布尔值，表示当前节点是否在文档之中。

4.8.2 DOM 节点常用操作

createElement 方法: document.createElement(tagName)

创建元素节点，返回一个 Element 对象。tagName 用来指明 html 元素类型，如 p、div。需要和 appendChild()或 insertBefore()方法联合使用，将元素显示在页面中。

还可以创建文本节点、属性节点等。可以创建一个空的文档片段节点。文档片段节点可以用来存储临时的 DOM 节点，然后一次性插入到文档中。

appendChild 方法：

接受一个节点对象作为参数，将其作为最后一个子节点，插入当前节点。

insertBefore 方法：

用于将某个节点插入父节点内部的指定位置。接受两个参数，预插入节点、父节点内部的 `referenceNode` 子节点，返回值是插入的新节点。如果第二个参数为 `null`，则新节点将插在当前节点内部的最后位置，变成最后一个子节点。

上面的插入操作，如果参数节点是 DOM 已经存在的节点，会将其从原来的位置移动到新位置。如果参数是 `DocumentFragment` 节点，那么插入的是 `DocumentFragment` 的所有子节点。

由于没有 `insertAfter`，可以 `InsertBefore` 某个节点的 `NextSibiling`，模拟相同效果。

hasChildNodes 方法：

返回一个布尔值，表示当前节点是否有子节点，这里的子节点包括所有类型的节点。

判断一个节点有没有子节点有多种方法：

- `node.hasChildNodes();`
- `node.firstChild != null`
- `node.childNodes && node.childNodes.length > 0`

removeChild 方法：

接受一个子节点作为参数，用于从当前节点移除该子节点。返回值是移除的子节点。被移除的节点依然存在于内存之中，但不再是 DOM 的一部分。

replaceChild 方法：

用于将一个新的节点，替换当前节点的某一个子节点，返回值是替换走的那个节点 `oldChild`。

contains 方法：

返回一个布尔值，表示参数节点是否满足以下三个条件之一：

- 参数节点为当前节点
- 参数节点为当前节点的子节点
- 参数节点为当前节点的后代节点

compareDocumentPosition 方法：

用法与 `contains` 方法完全一致，返回一个七个比特位的二进制值，表示参数节点与当前节点的关系。

isEqualNode 方法:

返回一个布尔值,用于检查两个节点是否相等,所谓相等,指的是两个节点的类型相同、属性相同、子节点相同。

isSameNode 方法:

返回一个布尔值,表示两个节点是否为同一个节点。

getRootNode 方法:

返回当前节点所在文档的根节点,与 ownerDocument 属性的作用相同。

4.8.3 节点集合 NodeList 和 HTMLCollection

NodeList 可以包含各种类型的节点。

HTMLCollection 只能包含 HTML 元素节点。

(1) NodeList

以下方法可以得到 NodeList 实例:

- Node.childNodes

- document.querySelectorAll()等节点搜索方法

NodeList 实例很像数组,可以使用 length 属性和 forEach 方法。

```
var children = document.body.childNodes;
Array.isArray(children) // false
Children.length // 34
Children.forEach(console.log) // ...
```

item 方法:

接受一个整数值作为参数,表示成员的位置,返回该位置上的成员。

(2) HTMLCollection

HTMLCollection 是一个节点对象的集合,只能包含元素节点 (element),不能包含其他类型的节点。同样有 length 属性、item 方法。

namedItem 方法:

参数是一个字符串,表示 id 属性或 name 属性的值,返回对应的元素节点。如果没有对应的节点,则返回 null。

4.8.4 ParentNode 和 ChildNode 接口

如果当前节点是父节点,就会继承 ParentNode 接口。

只有元素节点(element)、文档节点(document)和文档片段节点 (document Fragment) 拥有子节点,继承 ParentNode 接口。

(1) ParentNode

children 属性:

返回一个 `HTMLCollection` 实例，成员是当前节点所有元素子节点，该属性只读。注意：`children` 属性只包括元素子节点，不包括其他类型的子节点（比如文本子节点）。

`firstElementChild` 属性、`lastElementChild` 属性、`childElementCount` 属性、`append` 方法、`prepend` 方法（加在第一个元素子节点的前面），看名字，不详细描述了。

（2）ChildNode

如果一个节点有父节点，那么该节点就继承了 `ChildNode` 接口。

- `remove` 方法用于从父节点移除当前节点
- `before` 方法/`after` 方法用于在当前节点的前面/后面，插入一个或多个同级节点或者文本。两者拥有相同的父节点。注意，该方法不仅可以插入元素节点，还可以插入文本节点
- `replaceWith` 方法使用参数节点，替换当前节点。参数可以是元素节点，也可以是文本节点

4.9 DOM 中的主要对象

4.9.1 Document 节点对象

`document` 节点对象代表整个文档，每张网页都有自己的 `document` 对象。

节点的集合属性：

提供一部分属性用于返回文档内部特定元素节点集合，**返回值是个 `HTMLCollection` 实例，集合都是动态的**，原节点任何变化会立刻反映在集合中。

- `document.links` 属性返回所有设定了 `href` 属性的及节点（不返回节点）
- `document.forms` 属性返回所有 `<form>` 表单节点
- `document.images` 属性返回页面所有 `<img>` 图片节点
- `document.embeds` 属性、`plugins` 属性返回所有 `<embed>` 节点
- `document.scripts` 属性返回所有 `<script>` 节点
- `document.styleSheets` 属性返回文档内嵌或引入的样式表集

提供一部分属性，用于访问文档静态信息：

- `document.documentURI` 属性和 `document.URL` 属性返回一个字符串，表示当前文档的网址
- `document.domain` 属性返回当前文档的域名，不包含协议和接口
- `document.lastModified` 属性返回当前文档最后修改时间的字符串
- `document.title` 属性返回当前文档的标题，该属性是可写的
- `document.characterSet` 属性返回当前文档的编码

提供一部分属性，用于访问文档状态信息：

- `document.hidden` 属性返回布尔值，表示当前页面是否可见
- `document.visibilityState` 返回文档的可见状态
- `document.readyState` 属性返回当前文档的状态

`document` 节点对象的方法：

`document.querySelector` 方法：

接受一个 CSS 选择器作为参数，返回第一个匹配该选择器的元素节点，如果没有则返回 `null`。

`document.querySelectorAll` 方法：

返回一个 `NodeList` 对象，包含所有匹配给定选择器的节点。

`document.getElementsByTagName` 方法：

搜索 HTML 标签名，返回符合条件的元素。返回值是 `HTMLCollection` 实例，可以实时反映 HTML 文档的变化。

`document.getElementsByClassName` 方法：

返回所有 `class` 名字符合指定条件的元素，元素的变化实时反映在返回结果中

`document.getElementsByName` 方法：

用于选择所有拥有 `name` 属性的 HTML 元素

`document.getElementById` 方法：

返回匹配指定 `id` 属性的元素节点。

4.9.2 元素节点（Element）对象及属性、样式操作

`Element.id` 属性：

返回指定元素的 `id` 属性，该属性可读写。

`Element.tagName` 属性：

返回指定元素的大写标签名，与 `nodeName` 属性的值相等

`className` 属性：

用来读写当前元素节点的 `class` 属性。它的值是一个字符串，每个 `class` 之间用空格分割。

`classList` 属性：

返回一个类似数组的对象，当前元素节点的每个 `class` 就是这个对象的一个成员。`classList` 对象有下列方法，实现对 `class` 的增删改查：

`add`、`remove`、`toggle`（如果参数不存在就加入，否则移除）、`contains`（检查是否包含）、`item`、`toString`

innerHTML 方法:

返回一个字符串, 等同于该元素包含的所有 HTML 代码, 该属性可读写, 能改写所有元素节点的内容, 常用来设置某个节点的内容, 包括<html>和<body>元素。

outerHTML 方法:

返回一个字符串, 表示当前元素节点的所有 HTML 代码, 包括该元素本身和所有子元素, 该属性可读写。

Element.attributes 属性:

返回一个类似数组的动态对象, 成员是该元素标签的所有属性节点对象, 属性的实时变化都会反映在这个节点对象上。这个属性视为元素对象独有的。

如何通过 JavaScript 操作 CSS?

(1) 基本方法——操作 HTML 元素的 style 属性

```
div.setAttribute(  
    'style',  
    'background-color:red;' + 'border:1px solid black;'  
);
```

等同于以下代码:

```
<div style=' background-color:red; border:1px solid black;' />
```

(2) 利用 StyleSheet 接口操作样式表文件

document 对象的 styleSheets 属性, 可以返回当前页面的所有 StyleSheet 实例 (即所有样式表), 一个类似数组的对象。

StyleSheet.cssRules 属性指向一个类似数组的对象 (CSSRuleList 实例), 每个成员是当前样式表的一条 CSS 规则, 可以通过 rules.item(index) 或者 rules[index] 拿到, CSS 规则的条数通过 rules.length 拿到。

StyleSheet.insertRule 方法、StyleSheet.deleteRule 方法。

4.9.3 Text 节点和 DocumentFragment 节点

文本节点代表元素节点和属性节点的文本内容。

(1) 文本节点

可以使用 Document 节点的 createTextNode 方法创建一个文本节点。

data 属性:

等同于 nodeValue 属性, 用来设置或读取文本节点的内容

编辑 Text 节点文本内容的方法:

- appendData(): 在 Text 节点尾部追加字符串
- deleteData(): 第一个参数为子字符串开始位置, 第二个参数为长度

- `insertData()`: 第一个参数为插入位置，第二个参数为插入的子字符串
- `replaceData()`: 第一个参数为替换开始位置，第二个参数为需要被替换掉的长度，第三个参数为新加入的字符串
- `subStringData()`: 第一个参数为子字符串在 `Text` 节点中的开始位置，第二个参数为子字符串长度

(2) `DocumentFragment` 节点

代表一个文档的片段，本身就是一个完整的 DOM 树形结构。它没有父节点。一般用于构建一个 DOM 结构，然后插入当前文档。

`DocumentFragment` 节点本身不能被插入当前文档，当它作为方法参数时，是它的所有子节点插入当前文档，而不是它自身。

一旦 `DocumentFragment` 节点被添加进当前文档，它自身就变成了空节点（`textContent` 属性为空字符串），可以被再次使用。

4.10 事件

JavaScript 的事件很多，主要包括如下几种：

- 鼠标事件
- 键盘事件
- 表单事件
- 编辑事件
- 页面相关事件

浏览器的事件模型，就是通过监听函数（`listener`）对事件做出反应。

DOM 的事件操作（监听和触发），都定义在 `EventTarget` 接口。该接口主要提供三个实例方法：

- `addEventListener`: 绑定事件的监听函数
- `removeEventListener`: 移除事件的监听函数
- `dispatchEvent`: 触发事件

一个事件发生后，会在子元素和父元素之间传播（propagation）。

事件一开始从文档的根节点流向目标对象（捕获阶段），然后在目标对象向上被触发（目标阶段），之后再回溯到文档的根节点（冒泡阶段）。

事件的代理（`delegation`）

由于事件会在冒泡阶段向上传播到父节点，因此可以把子节点的监听函数定义在父节点上，由父节点的监听函数统一处理多个子元素的事件。